

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 07 · LECTURE NOTES

Signatures and Certificates

Authenticity at scale, and the trust problem.

MODULE OUTCOMES

- Explain signatures and how they differ from MACs.
- Name the schemes and choose between them.
- Explain what a certificate is and what it asserts.
- Explain the CA trust model and its weaknesses.
- Explain revocation and why it does not really work.

1 Signatures

	MAC	Signature
Keys	One shared secret, held by both parties.	A private key signs; the corresponding public key verifies.
Who can verify	Only someone holding the key.	Anyone at all, including parties with no prior relationship.
Who can forge	Any keyholder — which includes the verifier.	Only the holder of the private key.
Non-repudiation	No. Either party could have produced the tag, so neither can prove the other did.	Yes, cryptographically — the signer cannot credibly deny having signed.
Speed	Very fast — a hash-rate operation.	Orders of magnitude slower, which matters at volume.

Non-repudiation is the property that only a signature provides, and it follows directly from the asymmetry: since the verifier cannot produce the signature, its existence proves the signer did. **And the practical consequence is the useful one: if you do not need public verification or non-repudiation, use a MAC** — it is faster, simpler, has no key distribution problem beyond the shared secret, and is harder to misuse. **Signatures are reached for far more often than they are needed.**

- **Ed25519. The default choice** — fast, 64-byte signatures, 32-byte keys, **deterministic (so there is no per-signature nonce to leak)**, and specifically designed to be difficult to misimplement.

- **ECDSA. Widely required by existing standards and genuinely hazardous: a repeated nonce, or even a slightly biased one, reveals the private key outright** — recoverable from two signatures. **This has happened in production repeatedly**, including in a games console's code signing and in cryptocurrency wallets.
- **RSA-PSS. Entirely fine, with adequate key sizes** — 3072 bits for roughly 128-bit security. **Prefer PSS over the older PKCS#1 v1.5 padding**, whose signature verification has produced forgery bugs in multiple implementations.
- **And the post-quantum signature schemes** (Module 12) — now standardised, with substantially larger signatures and keys, which is the cost being weighed.
- **Ed25519's determinism is the feature rather than an incidental property: no per-signature randomness means no randomness failure** (Module 02 §3), which removes the entire class of failure that makes ECDSA dangerous. **ECDSA's nonce requirement is the sharpest edge in deployed cryptography**, and it is why both deterministic nonce generation (RFC 6979) and Ed25519 exist.

2 Certificates

A CERTIFICATE IS A SIGNATURE OVER THE CLAIM “THIS KEY BELONGS TO THIS NAME”

It contains a public key, a subject name, a validity period, and a set of constraints — and a signature over all of that by an issuer. That is the whole structure.

So verifying a certificate establishes that the issuer asserted the binding — and establishes nothing whatsoever about whether the assertion is true, which is a question about the issuer's verification procedures, its incentives, and its competence.

And the chain terminates in a root certificate you trust because it shipped with your operating system or your browser — which is the actual root of trust, and it is a business, legal, and governance arrangement rather than a cryptographic one.

Which is Module 01 §2's third limitation made entirely concrete: cryptography moved the trust problem rather than solving it, and the certificate authority system is where it now lives — which is why §3's weaknesses are structural rather than fixable with better algorithms.

```
FOR EACH CERTIFICATE IN THE CHAIN:
    signature verifies under the issuer's public key
    the current time is within the validity period
    the issuer is permitted to issue (basic constraints
        CA:TRUE, and the path length respected)
    key usage permits this purpose
    the signature algorithm is one you still accept

AND FOR THE LEAF, ADDITIONALLY:
    the name matches what you asked for -- the hostname
    you INTENDED to reach, not the one the
    certificate happens to offer
    the chain terminates at a trusted root

AND REVOCATION, if you can check it at all (section 4).

HISTORICAL FAILURES: missing the CA:TRUE check let any
leaf certificate sign others. Skipping the name check
accepted any valid certificate for any site. Both
shipped widely, in major software.
```

The name check is the one most often omitted, and omitting it accepts a perfectly valid certificate issued for an entirely different host — so an adversary needs only a legitimate certificate for a domain they control. **Both historical failures are worth stating explicitly**, because they explain why every modern library validates by default and why disabling validation requires an explicit, conspicuous flag — and why that flag appearing in production code is a finding.

3 The trust model

ANY CA CAN ISSUE FOR ANY NAME, SO THE SYSTEM IS AS STRONG AS ITS WEAKEST CA

Your browser trusts on the order of a hundred root authorities, each able to delegate to intermediates — and any one of them can issue a technically valid certificate for any domain name in the world. There is no partitioning by name.

So the security of every site is bounded by the least careful authority in the store, rather than by that site's own practices — which is an unusual and genuinely uncomfortable property, and one that no amount of care by a site operator can improve.

And this has been exploited rather than merely theorised. Certificate authorities have been compromised and have issued fraudulent certificates for major services, which were then used in real interception of real users' traffic at national scale.

The mitigations are detection rather than prevention: Certificate Transparency requires every issued certificate to be logged publicly in append-only Merkle logs (Module 05 §4), so a fraudulent issuance can be noticed by the domain owner — after the fact. Which is CSCE 701 Module 09's argument arriving in a protocol: when prevention is structurally unavailable, build detection.

4 Revocation

- **Revocation lists grew too large to distribute usefully,** and were cached for long enough that they were frequently stale at exactly the moment they mattered.
- **Online status checking introduced a privacy leak and an availability dependency — the responder learns every site every user visits, and a responder outage would break the web** if clients depended on it.
- **So clients fail open: if the status check cannot be completed, the connection proceeds anyway —** which means **an adversary who can block the status check has thereby defeated revocation entirely,** and an adversary positioned to use a stolen certificate is by definition positioned to block a check.
- **Stapling improves matters by having the server present its own recent signed status,** which removes the privacy leak and the per-client dependency — and depends entirely on the server choosing to do it.
- **And the actual answer turned out to be shorter lifetimes: a certificate valid for weeks needs far less revocation than one valid for years,** because the exposure window closes on its own. **Which is why automated issuance mattered so much —** it made short lifetimes operationally possible. **Short lifetimes plus automation replaced revocation rather than fixing it:** an engineering answer to a problem that resisted a protocol answer for two decades.

WHAT TO TAKE FROM THIS MODULE

Use Ed25519 unless an external standard forces otherwise, and use a MAC whenever you do not actually need public verification or non-repudiation — which is considerably more often than people assume (§1).

Never implement certificate validation yourself. Use your platform's validator, and do not disable any of its checks — every historical failure in §2 was a skipped check, and a `verify=False` in a client library is a security decision being made without one being taken.

Understand that the trust root is social rather than mathematical — which is why pinning, transparency logs, and short lifetimes all exist, and why none of them is cryptography. The strongest cipher in the world sits behind a hundred organisations' issuance procedures.

And state the dependency when you make a claim: 'authentic, assuming no trusted certificate authority misissues' is the honest form — which Module 13 develops into the general practice, and which is the program's closing rule appearing here as a specific conditional.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Aumasson — Serious Cryptography, chapter 12	§1's schemes , with the ECDSA nonce hazard explained properly.
Ristic — Bulletproof TLS and PKI	§2 through §4 as the practical reference — the best single treatment of the certificate ecosystem. Library copy.
Certificate Transparency (free)	§3's mitigation , and worth reading as a design response to an unfixable trust model.
RFC 6979 — Deterministic ECDSA (free)	§1's fix for the sharpest edge — and a good example of removing a failure mode rather than warning about it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Decide for five uses in your own system** whether a MAC or a signature is correct, and justify each.
- 2 Sign the same message twice with Ed25519** and with ECDSA, and compare the outputs.
- 3 Explain why one is identical and one is not**, and what follows for randomness.
- 4 Dump a real certificate chain** with OpenSSL and read every field.

- 5 **Walk the chain by hand** against Part 2's validation list.
- 6 **Count the root authorities** your operating system trusts.
- 7 **Pick three and look up who operates them** and under which jurisdiction.
- 8 **Look up your own domain in a Certificate Transparency log** and check for anything unexpected.
- 9 **Determine whether your client fails open** on a failed revocation check.
- 10 **Find one `verify=False`** in code you have access to, and assess it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five differences between a MAC and a signature.
- 2 Which property is unique to signatures, and what follows practically?
- 3 Name four signature schemes and say which to prefer.
- 4 Why is ECDSA hazardous, and what are the two fixes?
- 5 What exactly does a certificate assert?
- 6 Why is the trust root not cryptographic?
- 7 Give the chain validation checklist, and the two historical failures.
- 8 State the structural weakness of the CA model.
- 9 Why does revocation fail, and what replaced it?
- 10 Give the honest form of a certificate-based authenticity claim.